SICP 期中考试讲解

王锦朋

NJU SICP

2025 年 11 月 14 日

背锅时刻

我负责...

1. P3: Lists
2. P6: Check-GPT

目录

P3: Lists

P6: Check-GPT

总结

命题意图：偷鸡线代作业

方阵是 n 行 n 列的一组数字

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}$$

命题意图：偷鸡线代作业

方阵是 n 行 n 列的一组数字

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}$$

矩阵可以表示成 List of Lists：

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
mat = [[A11, ..., A1n],
       [..., ..., ...],
       [An1, ..., Ann]]
```

命题意图：偷鸡线代作业

方阵是 n 行 n 列的一组数字

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}$$

矩阵可以表示成 List of Lists：

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
mat = [[A11, ..., A1n],
       [..., ..., ...],
       [An1, ..., Ann]]
```

$$A_{i,j} \Longleftrightarrow \texttt{mat[i][j]}$$

命题意图：偷鸡线代作业

方阵是 n 行 n 列的一组数字

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}$$

矩阵可以表示成 List of Lists：

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
1 mat = [[A11, ..., A1n],
2        [..., ..., ...],
3        [An1, ..., Ann]]
```

$$A_{i,j} \Longleftrightarrow \texttt{mat[i][j]}$$

$$n \Longleftrightarrow \texttt{len(mat)}$$

命题意图：偷鸡线代作业

方阵是 n 行 n 列的一组数字

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}$$

矩阵可以表示成 List of Lists：

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
1 mat = [[A11, ..., A1n],
2        [..., ..., ...],
3        [An1, ..., Ann]]
```

$$A_{i,j} \Longleftrightarrow \texttt{mat[i][j]}$$

$$n \Longleftrightarrow \texttt{len(mat)}$$

矩阵操作 $\Longleftrightarrow$ 输入为 List 的函数

3.1: 矩阵的迹

线性代数

矩阵的**迹**（trace）定义为对角线元素之和

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
1 [[A11, ..., A1n],
2  [..., ..., ...],
3  [An1, ..., Ann]]
```

3.1: 矩阵的迹

线性代数

矩阵的迹（trace）定义为对角线元素之和

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
1 [[A11, ..., A1n],
2  [..., ..., ...],
3  [An1, ..., Ann]]
```

List

$\mathrm{tr}(A) = \sum_{i=1}^{n} A_{i,i} = A_{1,1} + A_{2,2} + \cdots + A_{n,n}$

3.1: 矩阵的迹

线性代数

矩阵的迹（trace）定义为对角线元素之和

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
[[A11, ..., A1n],
 [..., ..., ...],
 [An1, ..., Ann]]
```

List

$$\mathrm{tr}(A) = \sum_{i=1}^{n} A_{i,i} = A_{1,1} + A_{2,2} + \cdots + A_{n,n}$$

```
trace(mat) = A11 + A22 + ... + Ann
```

3.1: 矩阵的迹

线性代数

矩阵的迹（trace）定义为对角线元素之和

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
1 [[A11, ..., A1n],
2  [..., ..., ...],
3  [An1, ..., Ann]]
```

List

$$\mathrm{tr}(A) = \sum_{i=1}^{n} A_{i,i} = A_{1,1} + A_{2,2} + \cdots + A_{n,n}$$

```
1 trace(mat) = A11 + A22 + ... + Ann
2            = mat[1][1] + ... + mat[n][n]
```

3.1: 矩阵的迹

线性代数

矩阵的迹（trace）定义为对角线元素之和

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
1 [[A11, ..., A1n],
2  [..., ..., ...],
3  [An1, ..., Ann]]
```

List

$$\mathrm{tr}(A) = \sum_{i=1}^{n} A_{i,i} = A_{1,1} + A_{2,2} + \cdots + A_{n,n}$$

```
1 trace(mat) = A11 + A22 + ... + Ann
2            = mat[1][1] + ... + mat[n][n]
3            = sum([mat[1][1], ..., mat[n][n]])
```

3.1: 矩阵的迹

线性代数

矩阵的迹（trace）定义为对角线元素之和

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
[[A11, ..., A1n],
 [..., ..., ...],
 [An1, ..., Ann]]
```

List

$$\mathrm{tr}(A) = \sum_{i=1}^{n} A_{i,i} = A_{1,1} + A_{2,2} + \cdots + A_{n,n}$$

```
trace(mat) = A11 + A22 + ... + Ann
           = mat[1][1] + ... + mat[n][n]
           = sum([mat[1][1], ..., mat[n][n]])
           = sum([mat[i][i] for i in range(n)])
```

3.1: 矩阵的迹

线性代数

矩阵的迹（trace）定义为对角线元素之和

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
[[A11, ..., A1n],
 [..., ..., ...],
 [An1, ..., Ann]]
```

List

$$\mathrm{tr}(A) = \sum_{i=1}^{n} A_{i,i} = A_{1,1} + A_{2,2} + \cdots + A_{n,n}$$

```
trace(mat) = A11 + A22 + ... + Ann
           = mat[1][1] + ... + mat[n][n]
           = sum([mat[1][1], ..., mat[n][n]])
           = sum([mat[i][i] for i in range(n)])
           = sum([mat[i][i] for i in range(len(mat))])
```

3.2: 矩阵的转置

线性代数

矩阵的**转置**（transpose）定义为元素沿对角线对称交换

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
1 [[A11, ... A1n],
2  [..., ..., ...],
3  [An1, ..., Ann]]
```

3.2: 矩阵的转置

线性代数

矩阵的**转置**（transpose）定义为元素沿对角线对称交换

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
1 [[A11, ... A1n],
2  [..., ..., ...],
3  [An1, ..., Ann]]
```

List

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}^{\mathsf{T}} = \begin{bmatrix} A_{1,1} & \cdots & A_{n,1} \\ \vdots & \ddots & \vdots \\ A_{1,n} & \cdots & A_{n,n} \end{bmatrix}$$

3.2: 矩阵的转置

线性代数

矩阵的**转置**（transpose）定义为元素沿对角线对称交换

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
[[A11, ... A1n],
 [..., ..., ...],
 [An1, ..., Ann]]
```

List

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}^{\mathsf{T}} = \begin{bmatrix} A_{1,1} & \cdots & A_{n,1} \\ \vdots & \ddots & \vdots \\ A_{1,n} & \cdots & A_{n,n} \end{bmatrix}$$

```
transpose(mat) = [[A11, ..., An1], ..., [A1n, ..., Ann]]
```

3.2: 矩阵的转置

线性代数

矩阵的**转置**（transpose）定义为元素沿对角线对称交换

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
[[A11, ... A1n],
 [..., ..., ...],
 [An1, ..., Ann]]
```

List

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}^{\mathsf{T}} = \begin{bmatrix} A_{1,1} & \cdots & A_{n,1} \\ \vdots & \ddots & \vdots \\ A_{1,n} & \cdots & A_{n,n} \end{bmatrix}$$

```
transpose(mat) = [[A11, ..., An1], ..., [A1n, ..., Ann]]
               = [[A1i, ..., Ani] for i in range(n)]
```

3.2: 矩阵的转置

线性代数

矩阵的**转置**（transpose）定义为元素沿对角线对称交换

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
[[A11, ... A1n],
 [..., ..., ...],
 [An1, ..., Ann]]
```

List

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}^{\mathsf{T}} = \begin{bmatrix} A_{1,1} & \cdots & A_{n,1} \\ \vdots & \ddots & \vdots \\ A_{1,n} & \cdots & A_{n,n} \end{bmatrix}$$

```
transpose(mat) = [[A11, ..., An1], ..., [A1n, ..., Ann]]
               = [[A1i, ..., Ani] for i in range(n)]
               = [[Aji for j in range(n)] for i in range(n)]
```

3.2: 矩阵的转置

线性代数

矩阵的**转置**（transpose）定义为元素沿对角线对称交换

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
[[A11, ... A1n],
 [..., ..., ...],
 [An1, ..., Ann]]
```

List

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}^{\mathsf{T}} = \begin{bmatrix} A_{1,1} & \cdots & A_{n,1} \\ \vdots & \ddots & \vdots \\ A_{1,n} & \cdots & A_{n,n} \end{bmatrix}$$

```
transpose(mat) = [[A11, ..., An1], ..., [A1n, ..., Ann]]
               = [[A1i, ..., Ani] for i in range(n)]
               = [[Aji for j in range(n)] for i in range(n)]
               = [[mat[j][i] for j in range(n)] for i in range(n)]
```

3.2: 矩阵的转置

线性代数

矩阵的**转置**（transpose）定义为元素沿对角线对称交换

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix} \Longleftrightarrow$$

```
[[A11, ... A1n],
 [..., ..., ...],
 [An1, ..., Ann]]
```

List

$$\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ \vdots & \ddots & \vdots \\ A_{n,1} & \cdots & A_{n,n} \end{bmatrix}^{\mathsf{T}} = \begin{bmatrix} A_{1,1} & \cdots & A_{n,1} \\ \vdots & \ddots & \vdots \\ A_{1,n} & \cdots & A_{n,n} \end{bmatrix}$$

```
transpose(mat) = [[A11, ..., An1], ..., [A1n, ..., Ann]]
               = [[A1i, ..., Ani] for i in range(n)]
               = [[Aji for j in range(n)] for i in range(n)]
               = [[mat[j][i] for j in range(n)] for i in range(n)]
               = [[mat[j][i] for j in range(len(mat))]
                             for i in range(len(mat))]
```

还可以试试...

怎么实现（不一定用 List Comprehension）

1. 矩阵加法？
2. 矩阵乘法？
3. 行列式？
4. 矩阵求逆？

3.3: List Diagram

易错点

1. 可变数据: `silver` 指向 `ones`
2. 切片产生新 List:
   `silver[:0]` 是空的 `[]`
   `silver[1:]` 是浅拷贝（Shallow Copy）
3. 拼接产生新 List:
   `silver[:0] + silver[1:]`
   ⇒ `[] + silver[1:]`
   ⇒ `silver[1:]`

目录

P3: Lists

P6: Check-GPT

总结

命题意图：怎么聪明地 Debug

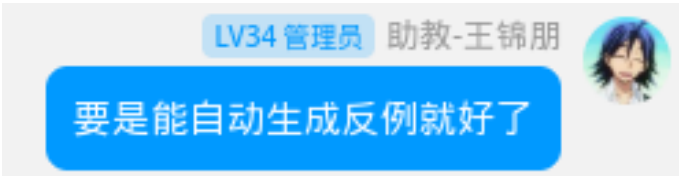
作为助教，我会...

1. 收到很多很多提问（请问我哪里错了）
2. 看很多很多的代码（请帮我 Debug）
3. 构造很多很多反例（“如此这般你的程序就不对了”）

命题意图：怎么聪明地 Debug

作为助教，我会...

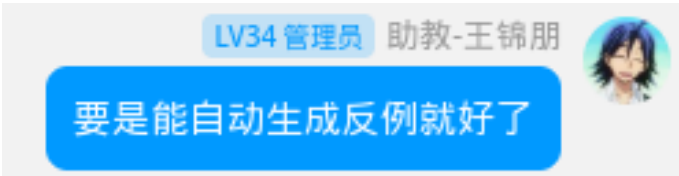
1. 收到很多很多提问（请问我哪里错了）
2. 看很多很多的代码（请帮我 Debug）
3. 构造很多很多反例（“如此这般你的程序就不对了”）



命题意图：怎么聪明地 Debug

作为助教，我会...

1. 收到很多很多提问（请问我哪里错了）
2. 看很多很多的代码（请帮我 Debug）
3. 构造很多很多反例（“如此这般你的程序就不对了”）

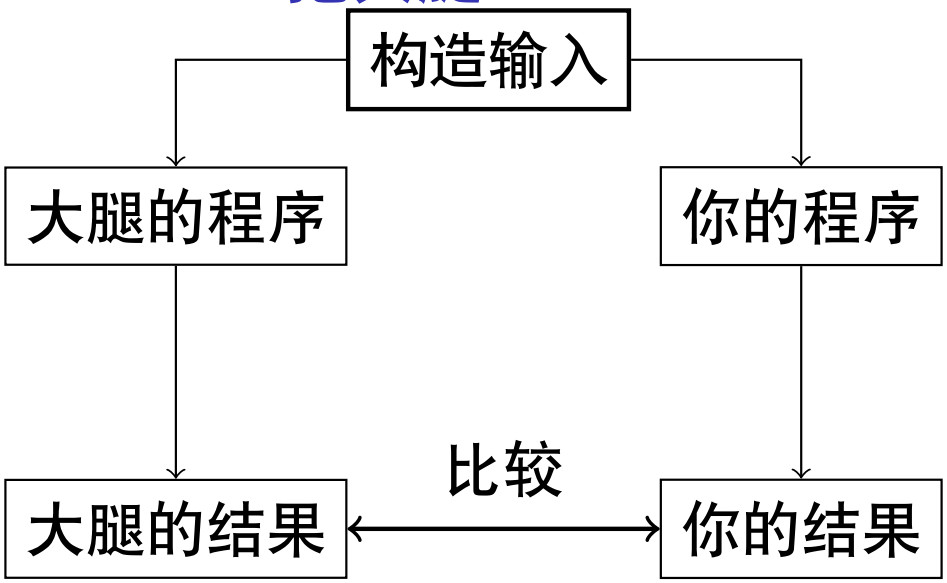


“做成阅读题，但是送点分吧”

好像变成了送命题

命题意图：怎么聪明地 Debug

DiffTest：抱大腿[1]



[1] 大腿可以是助教、室友、群友、生成式 ai

6.1: Why We Need Check-GPT

本题出现的程序均来自 OJ 真实提交

易错点

1. 本题的 `number_of_k(0, 0)` 会返回错误值 `0`
2. `'2020' + '5'` 的结果带引号 `'20205'`
3. `tree(2025)`, not `[2025]`, not `2025`

得分	
评分人	

6 Check-GPT: Generate Powerful Tests (18 pts)

You may need the following tree APIs in this problem:

- `tree(label, branches=[])`
- `label(tree)`
- `branches(tree)`
- `copy_tree(tree)`
- The interactive output of a tree should be the simplest expression that, when evaluated, reconstructs the same tree.

4. **Error**: `0 + 'yay'`

6.2.1: Generating Pairs

逻辑合理的都给了分

generate_n

```
while len(inputs) < n:
    if valid(current):
        inputs.append(current)
    current = f(current)
```

```
while n > 0:
    if valid(current):
        inputs.append(current)
    n, current = n - 1, f(current)
```

generate_n_k

```
generate_n(0, lambda x: x + 1, max_n, lambda x: True)
generate_n(max_n - 1, lambda x: x - 1, max_n, lambda x: True)
```

6.2.1: Generating Pairs

逻辑合理的都给了分

generate_n

```
while len(inputs) < n:
    if valid(current):
        inputs.append(current)
    current = f(current)
```

```
while n > 0:
    if valid(current):
        inputs.append(current)
    n, current = n - 1, f(current)
```

generate_n_k

```
generate_n(0, lambda x: x + 1, max_n, lambda x: True)
generate_n(max_n - 1, lambda x: x - 1, max_n, lambda x: True)
```

易错点

1. generate_n 应当返回**恰好** n 个元素（< vs <=）
2. 不少同学写了 f(start)。这么生成的 List 长啥样？
3. “Your answer must be a single call to generate_n”

6.2.2: Generating Trees

you might have guessed, binary trees are also recursively defined, just as the parenthesis sequence:

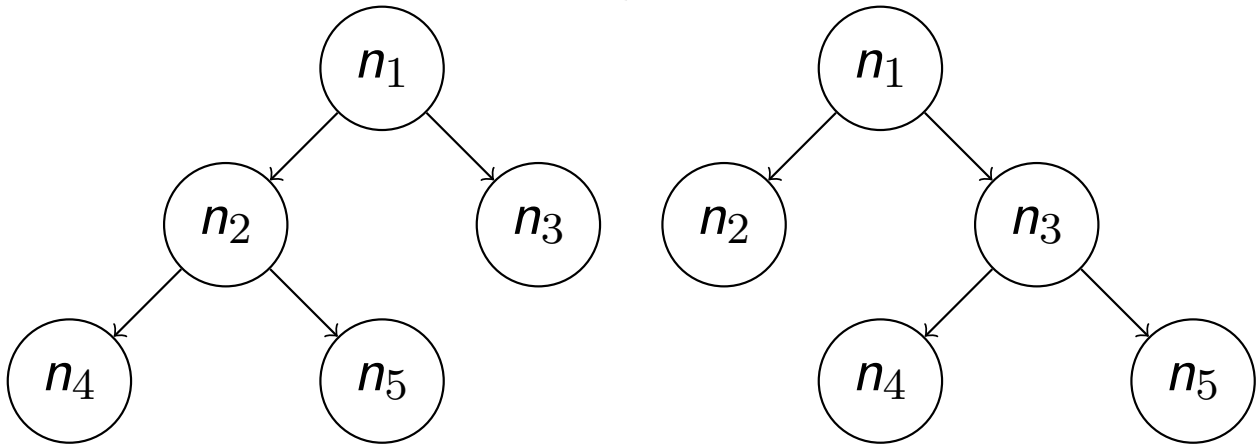
- For all valid labels, `tree(l)` is a binary tree.
- If `left` and `right` are binary trees, then for all valid labels `l`, `tree(l, [left, right])` is a binary tree.

```
1  def generate_trees(size, labels):
2      if size == 0:
3          return []
4      elif size == 1:
5          return [tree(l) for l in labels]
6      else:
7          result = []
8          for l in labels:
9              for k in range(size):
10                 left_subtrees = generate_trees(k, labels)
11                 right_subtrees = generate_trees(size - k - 1, labels)
12                 for left in left_subtrees:
13                     for right in right_subtrees:
14                         result.append(tree(l, [left, right]))
15         return result
```

6.2.2: Generating Trees

`generate_trees(5, ['sicp', '2025'])`

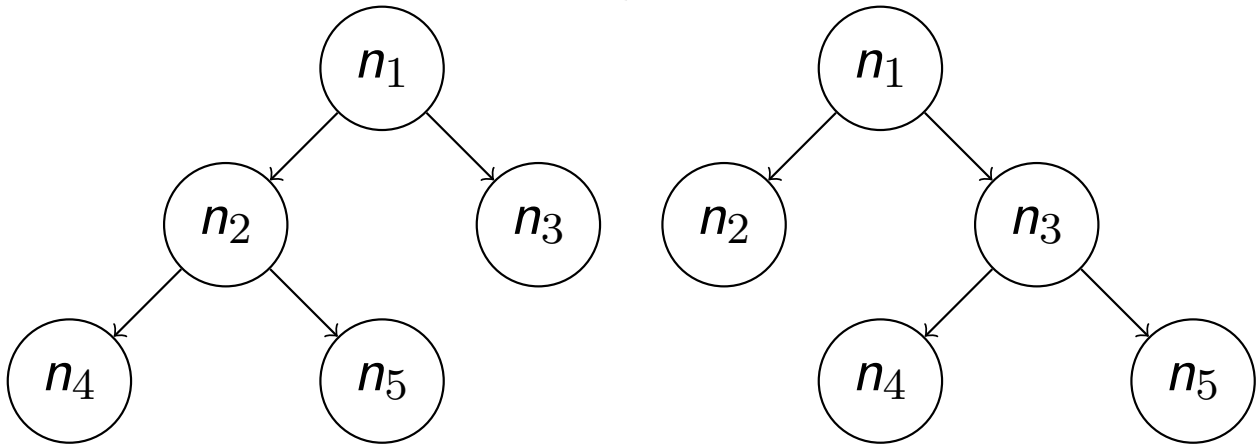
1. 恰好 5 个 label 的 binary tree 有 2 种:



6.2.2: Generating Trees

`generate_trees(5, ['sicp', '2025'])`

1. 恰好 5 个 label 的 binary tree 有 2 种:

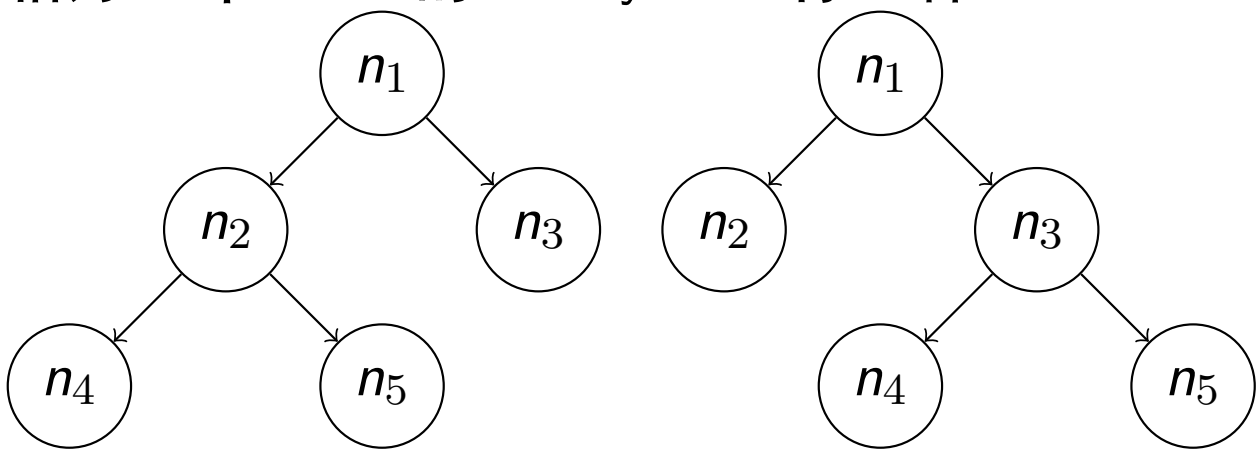


2. 每个节点有 2 种可能，`'sicp'` 或者 `'2025'`

6.2.2: Generating Trees

`generate_trees(5, ['sicp', '2025'])`

1. 恰好 5 个 label 的 binary tree 有 2 种:



2. 每个节点有 2 种可能，`'sicp'` 或者 `'2025'`
3. 总共 $2 \times 2^5 = 2^6 = 64$

6.3: Testing With Generated Inputs

1. `expected, got = number_of_k_steal_chicken(n, k), number_of_k(n, k)`

6.3: Testing With Generated Inputs

1. `expected, got = number_of_k_steal_chicken(n, k), number_of_k(n, k)`
2. 不存在 4 个 label 的 binary tree（why?），因此 `generate_trees(4, ['S', 'I', 'C', 'P'])` $= 0$

to see if it is robust enough to function correctly, without raising Errors. However, if we take a closer look at the test we have just defined above, it actually does not generate many trees under some special `size`. For example, `generate_trees(4, ['S', 'I', 'C', 'P'])` will return a list containing _____ trees (1pts).

6.3: Testing With Generated Inputs

1. `expected, got = number_of_k_steal_chicken(n, k), number_of_k(n, k)`
2. 不存在 4 个 label 的 binary tree（why?），因此 `generate_trees(4, ['S', 'I', 'C', 'P']) = 0`

   to see if it is robust enough to function correctly, without raising Errors. However, if we take a closer look at the test we have just defined above, it actually does not generate many trees under some special `size`. For example, `generate_trees(4, ['S', 'I', 'C', 'P'])` will return a list containing _____ trees (1pts).

3. 用 sum：

```
1 sum([generate_trees(s, labels) for s in range(size)], [])
```

   不用 sum：

```
1 [t for s in range(size) for t in generate_trees(s, labels)]
```

   递归：

```
1 [] if size == 0 else
2       generate_trees(size, labels)
3     + generate_trees_at_most(size - 1, labels)
```

目录

P3: Lists

P6: Check-GPT

总结

总结

1. 多动动手
2. 英语阅读
3. 读懂别人的暗示